

# SABLE: a SAT solver where every answer carries a proof

Technical and commercial positioning

August 2026

## Abstract

SABLE is a conflict-driven clause-learning (CDCL) SAT toolkit whose defining property is not speed but *checkability*: every satisfiable answer is re-evaluated against the input formula, and every unsatisfiable answer produces a DRAT proof replayed by a checker that shares no code with the solver. Checking costs about what solving costs, so it is affordable to leave on. The Python core has zero third-party dependencies; an optional Rust engine is 18× faster and reproduces the Python engine’s search bit-for-bit.

On top of the solver sits an equivalence checker for a restricted subset of Python: give it two functions and it proves they agree on every input at a declared bit width, or hands back an input where they do not. Tests check the inputs you thought of; this checks all of them.

This document states what the tool is, what it measurably is and is not, where a certified SAT solver could earn its place commercially, and what remains to be built. It includes a section on experiments that were implemented, measured, and thrown away, because a positioning document that lists its own failures is one whose other claims can be believed.

## 1 The problem

SAT solvers answer one question: does an assignment satisfying this Boolean formula exist? The question is dull; the applications are not. Hardware equivalence checking, bounded model checking, planning, scheduling, package dependency resolution and cryptanalysis all reduce to it, and the reductions are routine enough that SAT has become infrastructure.

The two answers are not symmetric, and this asymmetry is the whole opportunity.

**SAT** hands you a satisfying assignment. Anyone can check it in linear time: substitute and evaluate. The solver could be riddled with bugs and you would still know the answer is right.

**UNSAT** hands you nothing. It is a claim that *no* assignment exists—a statement about an exponentially large space that the solver explored on your behalf and is now asking you to take on faith. And UNSAT is the answer that matters most: “these two circuits are equivalent”, “this state is unreachable”, “this configuration is impossible” are all UNSAT results, and they are what safety arguments are built on.

Modern solvers are large, aggressively optimised C/C++ programs. Bugs in them are found regularly. When a solver reports UNSAT and a certification package rests on that answer, “it did not crash” is a weak foundation.

## 2 What SABLE is

A SAT toolkit built so that no answer requires trust.

- **Every SAT answer is verified** against the input formula, clause by clause, before it is returned.
- **Every UNSAT answer emits a DRAT proof** — the standard certificate format — which an independent checker in the same package replays step by step, confirming each learnt clause really follows and that the empty clause is reached.

- **The checker shares no code with the solver.** It has its own propagation engine, clause store and watch scheme. A checker built from the solver’s own machinery would certify the solver’s bugs alongside its results.
- **There are two checkers, and they must agree.** A Python reference and a Rust implementation are both shipped, and the test suite runs both over every proof — valid and deliberately corrupted — requiring identical verdicts. A checker’s job is to disagree with a buggy solver, so having exactly one, written against the same mental model as the solver, is the case to distrust.
- **The Python core has no dependencies.** It runs anywhere Python 3.10 runs. The Rust engine is optional, opt-in, and never selected silently.

Beyond the solver: two DRAT checkers (RUP and RAT), SatELite-style preprocessing with model reconstruction, Tseitin and cardinality and pseudo-Boolean encodings, minimal-unsatisfiable-subset extraction, a finite-domain modelling layer, a parallel portfolio whose winning thread still emits a verifiable proof, and the Python equivalence checker of section 6.

### 3 Evidence, not assertion

The distinguishing claim is testability, so the evidence is the product.

- **Two independent implementations agree bit-for-bit.** Across 7 instances  $\times$  5 configurations, the Rust engine produces *identical* conflict counts to the Python one—and identical decisions, propagations and restarts. Not similar: identical. That catches the class of bug verdict agreement hides, such as a mis-ordered watch list or a different heap tie-break.
- **The checker verifies foreign proofs.** It has confirmed CaDiCaL refutations up to **109,179 steps**. A checker that only ever sees its own solver’s proofs could share a misconception with it; agreeing with an implementation that shares no code is different evidence.
- **The checker demonstrably rejects.** Corrupted proofs—injected clauses, flipped literals, reordered steps, truncations—are rejected, and every acceptance of a random clause was independently confirmed to be a genuine entailment. A verifier that has only ever said “yes” is indistinguishable from `return True`.
- **1000-instance end-to-end validation.** Preprocess, solve, reconstruct, prove, check, extract a minimal unsatisfiable subset — each cross-checked against exhaustive enumeration *and* an independently written DPLL. 400 SAT with models verified, 600 UNSAT with proofs and MUSes verified. Zero failures.
- **344 public SATLIB instances.** All verdicts matched the benchmark’s declared answer, and every solver on the machine agreed. The corpus was 164 until the two families that decide the outcome were raised to the 100 instances each SATLIB ships — at the smaller size only 14 instances ran long enough to time, and one of them decided the ranking.
- **Two checkers agree, including on rejections.** The Python and Rust checkers are run over every proof in the suite and must return identical verdicts, reasons and step counts — corrupted proofs included, since a fast checker that accepted everything would sail through a suite that only fed it valid ones.
- 263 automated tests, 76% statement coverage with an enforced floor, continuous integration across Python 3.10–3.14 on Linux and macOS, the native engine, an external solver, and the built wheels installed into a clean environment. The floor has earned itself twice: it failed a build when a 353-line module arrived under-tested, and again when this release’s error handling did.
- **The artefact is tested, not just the repository.** The wheels are installed into an

empty virtualenv and exercised from a directory with no checkout on the path, because a suite that imports from the source tree cannot tell you whether the package is complete. The pure-Python wheel is byte-identical between builds, checked in CI — a project arguing for verifiable provenance of answers should not ship artefacts whose own provenance is unverifiable.

## 4 Where it actually stands

Performance, measured on an Apple M3 Pro against every SAT solver installable from Homebrew, in the same session on the same instances. The figure is the *geometric mean of per-instance ratios*, computed only over instances the competitor spends more than 50 ms on — below that the measurement is process startup rather than solving. **A value above 1 means the competitor is faster.**

| Instances | SABLE engine         | CaDiCaL | CryptoMS | kissat      | MiniSat     | Z3    |
|-----------|----------------------|---------|----------|-------------|-------------|-------|
| own       | Python, 1 thread     | 17.22   | 6.44     | 18.26       | 27.82       | 18.77 |
| own       | Rust, 1 thread       | 0.94    | 0.38     | 1.01        | <b>1.55</b> | 1.05  |
| own       | Rust + preprocessing | 0.61    | 0.27     | 0.65        | 0.82        | 0.60  |
| own       | Rust, 5 threads      | 0.35    | 0.25     | 0.37        | 0.40        | 0.34  |
| SATLIB    | Rust + preprocessing | 0.83    | 0.26     | <b>0.84</b> | 0.88        | 0.43  |
| SATLIB    | Rust, 5 threads      | 0.47    | 0.17     | 0.74        | 0.56        | 0.29  |

Read the bold entries first.

SABLE is **not the fastest SAT solver**, and this document will not claim otherwise. The kissat column moved from 1.62 to 0.84 when the default changed to Luby restarts with target phases, and that number carries a caveat given its own subsection below. The bare engine loses to MiniSat by  $1.55\times$  — and MiniSat is from 2008 and implements essentially the same algorithms, which makes that gap the honest measure of the remaining implementation distance.

Two further cautions on the table. The five-thread rows are not like-for-like: they race five threads against single-threaded competitors. And the “own” rows are instances the author designed, which is exactly why the SATLIB rows exist — every ratio moved against SABLE when the instances came from somewhere else.

What the table does support: SABLE is in the same league as production solvers, from a codebase of roughly 8,700 lines with an executable specification in Python beside it.

### The kissat number, and why the short version of it is wrong

The public-corpus figure against kissat moved from 1.62 to 0.84. Three things have to be said alongside it, because each is the first thing a reader familiar with the field would ask.

**The corpus was enlarged before it was believed.** At its previous size only **14 instances** ran long enough to time, and dropping a single one of them moved the standing from second (1.105) to first (0.905). The 1.62 figure is the honest baseline that appeared once the two decisive families were raised to the 100 instances each that SATLIB ships; 203 instances now clear the threshold and the result survives removing the worst five.

**kissat has instance-class configurations, and against those the satisfiable half reverses.** Against `kissat -sat` the uf250 family goes from 1.03 to **1.28** — kissat wins it by 28%. Against `kissat -unsat`, uuf250 goes from 0.72 to 0.43 in SABLE’s favour. There is a real defence, namely that choosing between those flags requires knowing the answer, which is the question being asked, and that this is why competitions compare default configurations. It is a defence rather than a dismissal.

**The configuration was tuned on the corpus it is reported on.** That is training on the test set, and a held-out corpus is the only fix. Uniform random 3-SAT was also dropped from competition main tracks years ago as unrepresentative, and the hardest instance here takes three seconds against a competition timeout in the thousands.

So the defensible sentence is the long one — 203 uniform random instances at  $n = 250$ , single-threaded, against kissat in its default configuration, on one machine — and the short one, “faster than kissat”, is not.

## 5 Verification is now cheap enough to leave on

A certificate nobody checks is decoration. Checking used to cost far more than solving, which meant it was the first thing a user would switch off:

| Instance  | Solve    | Python checker | Rust checker | Check / solve                             |
|-----------|----------|----------------|--------------|-------------------------------------------|
| php(8,7)  | 22.9 ms  | 621.6 ms       | 40.0 ms      | $27\times \rightarrow \mathbf{1.7\times}$ |
| php(9,8)  | 167.5 ms | 4,132.8 ms     | 243.2 ms     | $25\times \rightarrow \mathbf{1.5\times}$ |
| php(10,9) | 2.16 s   | 102.8 s        | 6.9 s        | $48\times \rightarrow \mathbf{3.2\times}$ |

Checking now costs roughly what solving costs. Both checkers ship, and the faster one did not replace the slower one — the Python implementation is the independent opinion that makes agreement worth anything, and the suite runs both over every proof.

The parallel engine certifies too. Because portfolio threads share no learnt clauses, the winning thread’s proof is already a complete standalone refutation and is returned as-is. That was the payoff of a decision taken earlier on correctness grounds alone: refusing clause sharing costs some parallel speedup and buys a parallel solver whose answers remain checkable.

## 6 From SAT solver to a tool people use

The gap between “a good SAT solver” and “something a working engineer reaches for” is the encoding. `sable.pyeq` closes it for one question that comes up constantly:

*Did my refactor change behaviour?*

Give it two Python functions and a bit width. It compiles each from its AST into a fixed-width bit-vector circuit, wires the two into a miter, and either proves that no input distinguishes them or hands back one that does.

```
def slow(a, b): return a * 2 + b * 2
def fast(a, b): return (a + b) << 1
equivalent(slow, fast, widths={"a": 8, "b": 8}).proved    # True
```

That `True` covers all 65,536 input pairs. A unit test covers the ones someone thought of.

And it is not the solver’s opinion. The equivalence is an UNSAT result, so it emits a DRAT proof — 854 steps for the pair above — which the independent checker replays before the answer is allowed to say `True`. The result carries `proof_checked` and `proof_steps` to say so. This was not true in the previous revision of this document, and it was the single place in the toolkit where an answer still rested on trust.

A bounded check is a third possibility, and it is a distinct one: `max_conflicts` caps the search, and exhausting it yields `proved=None` — undecided — which is falsy but is not `False` and is emphatically not `True`. Collapsing “I gave up” into “I proved it” is the specific failure a verification tool cannot have, and there is a regression test whose only job is that distinction.

Run against deliberately plausible refactors, it finds:

- a clamp whose bounds checks were reordered — broken only when  $lo > hi$ , at  $x=0$ ,  $lo=16$ ,  $hi=-32$ ;
- $(a+b) \gg 1$  against  $a + ((b-a) \gg 1)$  — differing at  $a=-128$ ,  $b=-2$ , which is precisely the overflow the second form exists to avoid.

Three design commitments make the answers worth having. **Unsupported syntax is rejected loudly**, with a line number — `while` loops, unbounded `range`, division by a non-power-of-two, function calls. A checker that silently approximates a construct makes every “equivalent” answer meaningless, because you can never tell whether the proof covered your code or a simplification of it.

And **the semantics are stated rather than assumed**. Python integers are arbitrary precision; this models fixed-width two’s complement at the width you declare. So “equivalent” means equivalent *as machine integers*. When the circuits differ but Python agrees, the result says so explicitly:  $(x*4)//4$  equals  $x$  in Python and does not at 6 bits, because  $x*4$  wraps. “Your refactor is wrong” and “your refactor is fine until it overflows” are different findings and the tool distinguishes them.

Third, **the verified path refuses to preprocess**. Bounded variable elimination adds clauses that do not follow from the input formula alone, so a refutation of the preprocessed miter is not a refutation of the miter the caller asked about. Given a choice between a faster solve and a checkable one, this module takes the checkable one.

This is the piece with the clearest path to being something people pay for, and it is the newest and least proven. Its subset is small. Widening it — arrays, wider integers, calls — is ordinary work rather than research, because the hard part underneath is a solver that already produces checkable answers.

## 7 Why certification is the right axis

Competing on raw speed against CaDiCaL and kissat means competing with two decades of accumulated inprocessing engineering. That contest is not winnable and is not where the unmet need is.

The unmet need is **evidence**. In regulated hardware and software development — DO-178C in avionics, ISO 26262 in automotive, IEC 61508 in industrial control — a tool that participates in verification must itself be qualified, and the qualification argument is easier when the tool emits a checkable artefact than when it must be trusted wholesale. A solver that outputs a proof turns “trust this tool” into “check this proof”, which is a different and much cheaper conversation with an assessor.

Adjacent needs with the same shape:

- **EDA sign-off**. Equivalence checking answers UNSAT when two designs agree. That result gates tape-out.
- **Research reproducibility**. A published UNSAT result with a checkable proof is a stronger claim than one without.
- **Explaining failure**. When a configuration or schedule is infeasible, “it is impossible” is useless; *which constraints conflict* is actionable. SABLE extracts minimal unsatisfiable subsets and verifies their minimality.

None of this requires being the fastest solver. It requires being a solver whose answers can be checked, which is a property most fast solvers offer only partially and few offer end to end in one package.

## 8 Honest competitive assessment

|                       | <b>Production solvers</b><br>(CaDiCaL, kissat, CryptoMiniSat) | <b>SABLE</b>                                                                                      |
|-----------------------|---------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| Raw speed             | Faster, sometimes much                                        | Within 1–2× single-threaded; behind on the hardest instances                                      |
| Proof emission        | Yes (DRAT)                                                    | Yes (DRAT)                                                                                        |
| Proof <i>checking</i> | Separate tool (drat-trim)                                     | Included, two independent implementations that must agree, at roughly the cost of the solve       |
| Dependencies          | C/C++ toolchain                                               | None for the Python core                                                                          |
| Readability           | Tens of thousands of lines, heavily optimised                 | ~5,600 lines of documented Python as an executable specification                                  |
| Inprocessing          | Extensive                                                     | Deliberately minimal, having measured that the usual techniques did not pay here                  |
| Reach beyond CNF      | DIMACS in, verdict out                                        | Python functions in: prove two agree on every input at a width, or get the input that breaks them |
| Maturity              | Decades, competition-hardened                                 | Young; validated but not battle-tested                                                            |

The honest summary: production solvers are better solvers. SABLE is a better *explainer*, and a substantially better starting point for anyone who needs to understand, modify, or qualify what the solver did.

### The comparison that actually costs something

The table above compares against production solvers, which is the easy comparison. The hard one is against the tools that make the *same* claim this document makes, and against several of those SABLE loses outright. Sorted by what a user’s trust ultimately rests on:

| Level | Trust rests on                                                         | Representative                                                                              |
|-------|------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| 0     | the solver not having crashed                                          | MiniSat, CryptoMiniSat, Z3 — as installed on the test machine, none advertises proof output |
| 1     | a proof, replayed by an unverified checker                             | CaDiCaL or kissat, plus drat-trim                                                           |
| 1.5   | a proof, replayed by two independent implementations required to agree | <b>SABLE</b>                                                                                |
| 2     | a proof, replayed by a <i>machine-verified</i> checker                 | CaDiCaL → LRAT → cake_lpr                                                                   |
| 3     | a solver whose own correctness is machine-verified                     | IsaSAT, CreuSAT, versat                                                                     |

**SABLE is not at the top of that table and should not pretend to be.** cake\_lpr carries a correctness proof reaching down to its machine code; SABLE’s checkers are *tested*. Those are different epistemic categories and no quantity of tests closes the distance. IsaSAT goes further again and verifies the solver itself, so no certificate is required at all.

There is a sharper form of the objection, worth raising here rather than leaving for a reader to find: SABLE’s two checkers were written by one person. That is independence of *implementation*, not independence of *mind*. A shared misconception about DRAT semantics would sit in both. What the arrangement does catch is the ordinary implementation bug — and it demonstrably rejects corrupted proofs — but it is not verification and is not described as such anywhere in this document.

## So what is left, and it is not a guarantee

Every tool at levels 2 and 3 takes CNF. So does every tool at levels 0 and 1. The verified ecosystem has proved the last mile to a very high standard and left the first mile untouched:

*A verified solver gives you a verified answer to whichever question your encoder happened to ask.*

That is the gap. Nobody has verified the translation from a problem someone actually has into the clauses a solver consumes, and in practice that translation is hand-written, unreviewed, and the single most likely place for the whole chain to be wrong. Reaching level 2 today is an integration project: a solver, an LRAT toolchain, a checker extracted from a proof assistant, and your problem in DIMACS — and the last of those four is the one that never gets done.

SABLE’s claim is therefore narrow and about adoption rather than strength. It is the shortest path from a problem someone has to an answer they can check: one install, one call, and a proof already replayed by the time the verdict arrives, at roughly the cost of the solve. Two things on that path have no equivalent at any level of the table — a parallel engine whose answers remain checkable, and a front end that accepts Python functions rather than clauses.

The resolution is not to compete with `cake_lpr` but to feed it. The proofs are already standard DRAT; emitting LRAT is item 1 of the roadmap in section 10, and it turns the pitch from “trust our checkers” into something considerably better defended: your function in, a proof out, checkable here for free or by a machine-verified checker when an assessor requires one.

## 9 Experiments that failed

Four optimisations were implemented in full, measured, and removed. They are listed because a positioning document that reports only its wins is not evidence of anything.

| Change                         | Expectation                      | Measured                          |
|--------------------------------|----------------------------------|-----------------------------------|
| Binary-clause specialisation   | Large; pigeonhole is 97% binary  | 1.5%, inside noise                |
| Vivification (inprocessing)    | Standard technique, expected win | Helped 1 of 6 seeds; also crashed |
| <code>target-cpu=native</code> | Free win on Apple silicon        | 1.3% <i>slower</i>                |
| Profile-guided optimisation    | Best remaining compiler lever    | 1.2% <i>slower</i>                |

All four have a common explanation, and finding it was worth more than any of them would have been: **a CDCL solver is bound by memory latency, not instruction throughput**. The hot loop chases watch lists and clause offsets through an arena far larger than L1 cache. Instruction selection, branch layout and a fast path that skips an already-cheap check do not address a cache miss.

A fifth correction is worth recording. Preprocessing was measured at 1.35× on the author’s structured instances and written up as “worth 1.3–1.5× on structured instances”. The public

corpus falsified it: preprocessing loses on *every* SATLIB family, including graph colouring and planning, where it removes a third of the clauses and still loses because those instances solve in milliseconds. The correct rule is that preprocessing pays when an instance is both structured *and* slow.

## 10 Road ahead

### Done since the previous revision

The three items at the head of the last roadmap are complete: the native proof checker (section 5), proofs from the threaded engine, and the Python equivalence checker (section 6). They are listed here because a roadmap that never records completion is a wish list.

### Technical, next

1. **Emit LRAT.** The single highest-value item here, because it changes what this project is competing with. A DRAT proof must be re-derived by the checker; an LRAT proof carries the antecedent clause indices, so checking becomes a linear scan — and, decisively, LRAT is what the *formally verified* checkers consume. `cake_lpr` is extracted from CakeML with a correctness proof reaching down to the machine code, and CaDiCaL already emits LRAT natively (`-checkproof=2`). Adding it stops SABLE being a weaker alternative to that ecosystem and makes it a front end for it: your Python function in, a proof out, checkable here for free or by a machine-verified checker when an assessor requires one.
2. **Widen the equivalence checker's subset.** Fixed-size arrays, multiple widths in one function, and calls to other supported functions. Each is ordinary compiler work; none needs research. This is where the tool becomes useful to people who are not already interested in SAT.
3. **Overflow-aware equivalence.** Rather than only reporting that a difference is an overflow artefact, prove the stronger property: these functions agree *and* neither overflows at this width. That is the answer most people actually want.
4. **Instance-aware configuration.** The portfolio and its configuration diversity already exist; selecting among them from cheap syntactic features is the natural next step. It needed a real instance corpus first, which now exists.
5. **Close the single-threaded gap to kissat and MiniSat.** The honest measurement in section 4 says this is where the remaining engineering distance is. Four attempts at it have already been measured and rejected (section 9), so the next one should start from a profile rather than from a technique.

One item is not technical and gates the section below: **the licence**. Nothing can be released publicly until it is chosen, and it should not be chosen until someone outside this repository has tried to use the tool. The options and what each costs are set out in `docs/LICENSING.md`; the current position — no licence, no rights granted — is the one that keeps all of them reachable.

### Commercial, as options rather than decisions

No business exists around this tool today, and this section does not pretend otherwise. Three shapes are plausible, in increasing order of commitment.

All three presuppose a licence, and there is not one. The repository grants no rights at all — deliberately, because a licence can be granted at any later date on any terms, while a permissive one cannot be withdrawn once released. The tree previously carried a CC0 public-domain dedication, inherited from the scratch phase, which is the single option with no path



back; it was removed before the first public release made it permanent. So the options below are live in the sense that any of them is still reachable, and none of them has been chosen.

### Open with support

Released permissively; the commercial layer is integration help, priority fixes, and assistance producing qualification evidence for an assessor. Lowest commitment, and it matches where the value already is: the artefacts, not the binary. It also forecloses charging for the code itself, permanently.

### Open core

Core solver released, the paid layer being the qualification package — traceability from requirements to tests, coverage evidence, tool-qualification documentation. This is what regulated customers actually buy, and it is work rather than code. It requires sole copyright to be maintained indefinitely, which is why contributions need an assignment.

### Verification as a service

The equivalence checker as a hosted CI check: push a refactor, get a proof it is behaviour-preserving or a failing input. The most product-shaped of the three, and no longer hypothetical — `sable.pyeq` does this today for a restricted subset of Python. What stands between the current state and a product is subset coverage rather than any unsolved problem, which is a pleasant kind of gap to have.

The honest prerequisite for any of them is the same: the tool must be fast enough not to be dismissed, and the certification story must be complete rather than partial. Both are now true, and as of this release it is also installable rather than merely runnable from a checkout.

What is not yet true is that anyone outside this repository has tried to use it. That is the next thing worth finding out, and it is also the thing the licence decision should wait on: choosing terms to fit a business model that may not exist is optimising against a guess. Silence costs nothing in the meantime.

## 11 Summary

SABLE is a SAT toolkit that treats an unverifiable answer as an incomplete one. It is not the fastest solver available and does not claim to be. It is a solver that hands you a proof, checks that proof with two independent implementations that must agree, and has been validated against exhaustive enumeration, an independent DPLL, five external solvers, and 164 public benchmark instances with known answers. Checking now costs about what solving costs, including on the parallel path, so the certificate is a default rather than a debugging option.

On top of that sits the beginning of an answer to a question working engineers actually ask — “did this refactor change behaviour?” — answered over the whole input space at a declared width instead of over the cases someone happened to write a test for.

Its most unusual property may be the one this document has been demonstrating throughout: the measurements that contradicted the author’s expectations were kept, and the claims they falsified were withdrawn.

---

Repository: [github.com/carlok/sable](https://github.com/carlok/sable) (private, and carrying no licence — no rights are granted pending that decision; the options and their costs are set out in `docs/LICENSING.md`). All figures in this document are reproducible with `make gate`, `make compare` and `make bench-public`. Public benchmark instances are from SATLIB (Hoos and Stützle, 1998).